

# ENSIA Impact Chatbot: A Retrieval-Augmented Generation System for Telegram Knowledge Base Exploration

By: Mohamed Aymen BENMALEK

June 18, 2026

## Abstract

This report presents the design and implementation of the ENSIA Impact Telegram chatbot, a Retrieval-Augmented Generation (RAG) system specialized for querying archived Telegram group conversations. The system supports multimodal attachment processing through EasyOCR and PyMuPDF, conversational context reconstruction using reply tracking, character-level corpus chunking, and semantic retrieval using BGE-M3 embeddings stored in Qdrant. A multilingual Qwen 2.5 7B model is employed for query reformulation and response generation in both Arabic and English. Additionally, an incremental knowledge extension pipeline enables efficient ingestion of newly exported chat data while preserving contextual continuity through boundary-overlap chunk generation.

## 1 Introduction

Telegram group chats contain a significant amount of institutional knowledge, including announcements, internship opportunities, event information, discussions, and shared documents. Traditional keyword-based search mechanisms often fail to retrieve relevant information due to linguistic variations and conversational context.

Retrieval-Augmented Generation (RAG) systems address this limitation by combining semantic retrieval with Large Language Models (LLMs). However, Telegram archives introduce additional challenges:

- **Multimodal Content:** Important information may be contained within images and PDF documents.
- **Conversational Context:** Messages are short, fragmented, and frequently reference previous messages through Telegram reply chains.

- **Multilingual Queries:** Users may ask questions in Arabic or English.
- **Continuous Growth:** Chat archives evolve over time and require efficient incremental updates.

The ENSIA Impact chatbot addresses these challenges through an optimized preprocessing, indexing, retrieval, and generation pipeline.

## 2 Data Preprocessing and Attachment Extraction

The raw data source consists of Telegram JSON exports. The preprocessing stage transforms these exports into a searchable corpus.

### 2.1 Message Filtering

System-generated events such as joins, leaves, and group modifications are removed. Only user-generated messages are retained.

### 2.2 Message Formatting

Each message is transformed into a structured format:

```
[FROM] Sender Name
[ON] 2025-01-01T12:00:00

Message content

[REPLY TO]
Parent message content

[ATTACHMENT_CONTENT]
Extracted attachment text
```

This structure preserves authorship, temporal information, reply relationships, and attachment content, allowing the retrieval system to capture conversational context more effectively.

## 2.3 Text Cleaning

The cleaning pipeline performs:

- Line-ending normalization
- Whitespace reduction
- Emoji removal
- Punctuation normalization
- Noise filtering

These operations improve embedding quality while preserving semantic content.

## 2.4 Attachment Processing

The chatbot supports multimodal content extraction.

### 2.4.1 Image OCR

Photo attachments are processed using EasyOCR configured for Arabic and English text extraction.

Extracted text is appended to the associated message under:

```
[ATTACHMENT_CONTENT]
Extracted text...
```

### 2.4.2 PDF Processing

PDF files are parsed using PyMuPDF.

For each page:

1. Attempt direct text extraction.
2. If no text exists, render the page as an image.
3. Apply EasyOCR to recover scanned content.

This approach supports both digital and scanned documents.

## 2.5 Reply Reconstruction

Telegram reply relationships are reconstructed using a hash map indexed by message identifiers.

When a message references another message through the `reply_to_message_id` field, the referenced message content is retrieved and appended under a dedicated `[REPLY TO]` block.

This strategy enables the embedding model to capture conversational context directly inside the chunk representation, improving retrieval quality for follow-up questions and fragmented discussions commonly found in Telegram group chats.

## 3 Corpus Construction and Chunking

After preprocessing, all messages are merged into a single chronological corpus.

Instead of fixed message windows, the system performs character-level chunking using LangChain's `RecursiveCharacterTextSplitter`.

Configuration:

- Chunk Size: 1500 characters
- Chunk Overlap: 300 characters

This strategy preserves context continuity while maximizing retrieval quality.

## 4 Embedding Generation and Vector Storage

Each generated chunk is converted into a dense vector representation using:

`BAAI/bge-m3`

BGE-M3 was selected because of its strong multilingual retrieval capabilities and support for Arabic and English text.

The generated embeddings are stored in a local Qdrant vector database:

```
./ensia_impact_qdrant_data
```

Metadata associated with each chunk includes:

- Keywords
- Main Topic
- Participants
- Link Presence

## 5 Knowledge Base Extension

Rebuilding the entire vector database whenever new chat exports become available is inefficient.

To solve this problem, the chatbot implements an incremental ingestion pipeline.

### 5.1 New Message Detection

The system compares:

- Previous Telegram export
- Newly exported Telegram data

Only messages with unseen message IDs are retained.

## 5.2 Boundary Context Preservation

To avoid context discontinuity between old and newly ingested data:

1. Retrieve the final messages from the previous export.
2. Append them to the beginning of the new corpus.
3. Perform chunking on the combined text.

As a result, the first new chunk naturally overlaps with the last historical chunk.

## 5.3 Database Update

New chunks are embedded and inserted into Qdrant:

```
vector_store.add_documents(  
    documents  
)
```

This allows continuous knowledge expansion without rebuilding the entire database.

# 6 RAG Pipeline

The chatbot uses a unified multilingual architecture based on the local Qwen 2.5 7B model deployed through Ollama.

## 6.1 Conversational Query Builder

Multi-turn conversations are converted into standalone search queries.

Conversation history is represented as:

```
[OLD]  
  
[QUERY]  
Previous Question  
  
[ANSWER]  
Previous Answer  
  
[NEW]  
Current Question
```

This enables context-aware retrieval for follow-up questions.

## 6.2 Semantic Retrieval

The generated search query is embedded using BGE-M3.

Qdrant retrieves the top-K matching chunks.

$$K = 10$$

## 6.3 Metadata-Based Reranking

Retrieved chunks are reranked using metadata overlap scores based on:

- Keywords
- Participants
- Organizations

This improves retrieval precision.

## 6.4 Response Generation

The reranked context is combined with the user query and passed to Qwen 2.5 7B.

The model generates a grounded answer constrained by retrieved evidence.

# 7 Software Architecture

The implementation follows modular software engineering principles.

## 7.1 Factory Pattern

The LLMFactory component encapsulates:

- Ollama model creation
- Configuration management
- Future model extensibility

## 7.2 Facade/Pipeline Pattern

The PipelineManager provides simplified execution interfaces:

```
run_full_pipeline()  
run_knowledge_extension()
```

These methods orchestrate the complete ingestion and retrieval workflow.

# 8 Conclusion

The ENSIA Impact chatbot demonstrates a production-oriented Retrieval-Augmented Generation architecture for Telegram knowledge exploration.

By combining OCR-based attachment extraction, PDF processing, conversational context reconstruction, multilingual semantic retrieval, incremental database updates, and local language model inference, the system provides a scalable

and continuously evolving platform for accessing institutional knowledge contained within Telegram archives.

Future work may include hybrid keyword-vector retrieval, reranking models, attachment image captioning, and distributed deployment.